科研小助手

面向科研场景的 AI 文献理解与写作 Agent

技术文档

第八届传智杯大赛 · 人工智能赛道

参赛作品技术文档

2026 年 10 月

一、作品概述

1.1 一句话定位

科研小助手是一个本地部署的文献理解与写作 Agent。用户把手里的 PDF 论文拖进来，系统完成细粒度解析、混合检索问答、中英双向翻译、引用关系图谱分析，并在写作环节提供受约束的框架生成、架构图绘制、数据可视化与中译英。全部功能围绕「让研究者少花时间在找和读，多花时间在想和写」这一个目标展开。

1.2 与赛题要求的对应

赛题将考察点拆为六大功能模块。下表给出模块、实现位置与实测证据的对应关系，所有数字均为本机真实库（39 篇 PDF）上的运行结果。

官方模块	实现文件	实测证据
1.1.1 论文去重	app/papers.py	文本相似度阈值 0.80；v1/v7 实判 0.873
1.1.2 细粒度解析	app/structure.py	39 篇 / 961 章节 / 282 表 / 287 图
1.2 知识问答	app/agent.py + app/rag/	逐句带页码引用；无证据时明确拒答
1.3 阅读与翻译	app/reader.py + app/translate.py	章节树、双向跳转、划词/悬停翻译
1.4 下载与推荐	app/online.py + app/recommend.py	arXiv + GitHub 两路检索，反馈调优
1.5 分析与生成	app/graph.py	36 节点 / 189 引用边
1.6 写作助手	app/writing.py 等 4 个模块	候选集约束生成、三渲染器、CSV 出图、中译英

1.3 工程规模

维度	数值
后端代码	12,548 行（app/）
前端页面	22,035 行（web/）
脚本与测试	2,874 行 + 5,066 行
REST 接口	33 个
测试用例	340 条（pytest 收集 369 个含参数化）

自动化检查	ruff 全绿 + 依赖自检 + CI 三步

二、系统架构

2.1 分层结构

系统采用「单进程 Web 服务 + 本地数据目录」的形态，不做分布式拆分。理由很实际：这是一份需要评委在浏览器里点得动的作品，多部署一个中间件只会增加演示失败的概率。

```
浏览器（原生 HTML/JS，无构建步骤）
        │  fetch / SSE
        ▼
FastAPI 应用 app/main.py  ── 33 个 REST 接口
        │
        ├─ app/agent.py     编排：规划→检索→抽取→校验→写作
        ├─ app/rag/         三路召回 + RRF 融合 + 交叉编码器重排
        ├─ app/graph.py     引用图谱与图谱挖掘算法
        ├─ app/writing.py   候选集约束生成
        ├─ app/chart.py / diagram.py / zh2en.py   写作三件套
        └─ app/structure.py / ocr.py / layout.py   解析层
        │
        ▼
data/   papers(原始 PDF) · chroma(向量) · meta(解析与索引) · models(本地模型)
```

2.2 数据流：一次问答发生了什么

- 前端 POST /chat/stream，服务端返回 SSE 事件流，逐条推送角色进度与证据。
- Planner 判定是单篇聚焦、多篇对比还是全库检索，并生成检索词。
- Retriever 走三路召回（稠密 / 哈希 / BM25），RRF 融合后取前 15 个候选。
- 交叉编码器对候选重排，保留相关性最高的若干片段作为证据。
- Extractor 在需要时补充结构化材料（论文卡片、表格行、章节原文）。
- Verifier 统计证据条数、覆盖论文数、可定位页码数，缺页码会标红。
- Writer 只依据上述证据撰写，每句标注 [来源: 文件 第 N 页]，证据不足则明确说明。

2.3 数据组织

路径	内容	是否入库
data/papers/	原始 PDF，只读，任何功能都不修改	否（体积大）
data/chroma/	向量索引，可用脚本重建	否

data/meta/structures.json	细粒度解析结果	是
data/meta/citation-graph.json	引用图谱，可用脚本重建	否
data/models/	本地 ONNX 模型（重排、Embedding）	否
data/catalog.json	论文元数据清单	是

「是否入库」这一列是刻意设计的结果：索引与图谱都能由脚本从原始 PDF 重建，放进仓库只会让克隆变慢且容易产生二进制冲突。原始 PDF 不进仓库则有另一个理由——公开仓库不应分发他人版权论文。

三、核心技术实现

赛题评分中「AI 技术深度」占 35%，原文列举了文档智能解析、高级 RAG 架构、多智能体协同三项。本章按这三项展开，并补充本地模型部署情况。

3.1 文档智能解析

3.1.1 四层降级策略

论文 PDF 的复杂度差异很大：有原生文本层、有双栏排版、有纯扫描件、有 541 页的书籍。系统按四层依次尝试，任一层失败就降级到下一层。

层级	触发条件	实现	代价
1 原生文本层	PDF 有可提取文本	pymupdf 按块提取	最快（毫秒级）
2 版式感知	检测到双栏	app/layout.py 按栏切分再排序	加几十毫秒
3 表格专项	页面含表格结构	pdfplumber 文本策略	较慢但准确
4 OCR 兜底	文本层为空（扫描件）	RapidOCR + ONNX	秒级，按页累计

实测覆盖：39 篇论文全部解析成功，其中包含扫描件 1706.03762v1_img.pdf 与 541 页的 Understanding Deep Learning。

3.1.2 结构化抽取的难点

真正花时间的地方不是提取文字，而是从版面里可靠地认出「哪一行是作者」。论文的作者区排版千奇百怪，实测踩到并被修复的五类失败模式：

- LaTeX 排版里作者之间没有逗号，只能靠机构标记切分。
- 名字后面紧跟脚注数字，如「Ashish Vaswani1」，不清掉数字就认不出名字边界。
- 姓名与上标编号粘连，如「Niki Parmar2,3Google Research」。
- 整行是机构署名而非人名，如「Google Brain」，会被误当成作者。
- 分组标题混在作者里，如 BLOOM 的「BigScience Workshop」「Major Contributors」不是人名；但「Llama Team, AI @ Meta」是机构作者，不能误删。

修复后作者抽取覆盖 38/39 篇，全部使用真实人名而非占位符。这条链路有独立回归测试 tests/test_authors.py（215 行）。

3.2 高级 RAG 架构

3.2.1 三路召回与融合

单路检索在学术文本上都有明显短板：稠密向量擅长语义但会漏专有名词，BM25 擅长精确匹配但不懂同义词，哈希向量作为轻量兜底。因此三路并行召回再融合。

通道	实现	权重	擅长
稠密	chromadb ONNXMiniLM_L6_V2（384 维）	2.0	语义相近但用词不同
哈希	自建 1024 维特征哈希	0.5	无模型时的兜底
BM25	自建（k1=1.2, b=0.6）	1.0	术语、公式号、人名精确命中

融合用 Reciprocal Rank Fusion，k 取 60：

```
fused[cid] += weight / (RRF_K + rank + 1)      # RRF_K 默认 60
```

权重可由环境变量 `RAG_W_DENSE` / `RAG_W_HASH` / `RAG_W_BM25` 覆盖

选 RRF 而不是加权分数相加的理由：三路通道的原始分数量纲完全不同（余弦相似度 0~1、BM25 无上界、哈希内积可正可负），直接相加等于让量纲大的通道主导结果。RRF 只用名次，天然规避了这个问题。

3.2.2 交叉编码器重排

召回阶段是「双塔」结构——问题和文档分别编码，速度快但精度有限。重排阶段换成「交叉编码器」：把问题与文档拼在一起送入模型，让它直接看两者的交互，精度高但慢，所以只用于重排前 15 个候选（RAG_RERANK_POOL）。

项	取值
模型	ms-marco-MiniLM-L-6-v2（ONNX 量化版，约 22 MB）
输入截断	512 token（BERT 类模型上限）
输出	logits（可正可负，只在批内可比）
缺模型时	抛 RerankUnavailable，调用方回退到 RRF 排序，检索不中断

关于输出是 logits 这件事，前端一度把它当成「融合分」显示，导致出现负数分数。已修正为按 score_type 区分显示：RRF 分叫「融合分」（恒正），重排分叫「重排分」（可为负，且只在同一问题的候选间可比）。

3.2.3 抗幻觉：候选集约束生成

这是本项目在写作环节最核心的一个设计决策，解决的问题是：大模型写综述时会凭空编造参考文献。常见做法是事后校验——生成完再检查引用是否真实，不合格就重写。本项目的做法是让编造在结构上不可能发生。

① 用 idea 检索真实文献，得到候选池，每篇分配 key：C1，C2，... Cn

② 候选池连同 key 一起塞进 prompt，明确要求：只能引用池内的 key

③ 模型只写标记 [C3]，不允许写出完整条目

④ 参考文献列表由代码从候选池取原始元数据生成，模型碰不到

⑤ 生成后校验：出现池外 key 即判失败

效果：实测生成 12 篇候选、正文引用 12 个 key，悬空 key 为 0。池子为空时系统不会硬生成，而是直接返回「没有检索到任何真实文献，为避免编造引用，未生成框架」。

同样的思路贯穿问答环节：证据不足时不编造，而是明确说明资料不足。实测提问「Attention 的核心机制是什么？」时，由于检索到的片段没有给出形式化定义，回答主动写明「检索片段只给出零散描述，没有 attention 的形式化定义……资料不足」。

3.3 多智能体协同

3.3.1 为什么不是四个模型互相聊天

调研阶段读了 MA-RAG、paper-qa、med-paper-assistant 等实现，主流做法是多个 Agent 通过共享状态或消息队列协作。但直接照搬会遇到一个硬约束：四角色互调意味着单次提问要多调 3~4 次大模型，现有单问耗时 6~20 秒，加倍后会超出演示可接受范围。

因此本项目采取「显式命名 + 可核验产物」的路线：把原本隐式串在一起的处理阶段显式建模成四个角色，每个角色留下可核验的产物与耗时，但不为了「像多智能体」而增加模型调用次数。

角色	职责	产物
Planner 规划	判定单篇/多篇/全库，生成检索词	计划类型 + 检索词
Retriever 检索	三路召回 + 融合 + 重排	召回片段数、覆盖论文数
Extractor 抽取	补充卡片/表格行/章节原文	结构化材料条数

Verifier 校验	统计证据与页码可定位性	证据数、页码完整性
Writer 写作	依据证据撰写并标注来源	字数、轮次

实测输出示例：

```
Status: 证据=4 · 覆盖论文=2 · 页码=2,11
Planner   0ms    全库检索 · 检索词「core mechanism of attention」
Retriever 2928ms 召回 4 段原文，覆盖 2 篇论文
Extractor 163ms  本次问题不需要结构化加工，直接用检索片段
Verifier  0ms    证据 4 条，其中 4 条可定位到具体页码
Writer    3337ms 生成答案（912 字，第 1 轮定稿）
```

这条时间线直接渲染在前端回答上方，用户可以看清系统做了什么、哪一步慢、证据够不够。Verifier 判定为失败时（证据缺页码、或完全没有证据）该行标红。

3.4 本地模型部署

评分标准原文写明「本地模型部署加分」。本项目在三个环节使用本地模型，均为 ONNX 运行时，不依赖网络：

用途	模型	体积	离线
文本嵌入	ONNXMiniLM_L6_V2（374 维）	约 80 MB	是
交叉编码器重排	ms-marco-MiniLM-L-6-v2 量化	22 MB	是
扫描件 OCR	RapidOCR（PP-OCR ONNX）	约 15 MB	是

只有最终撰写环节调用 DeepSeek API，且该调用可替换为任意 OpenAI 兼容端点（配置项 llm_base_url），演示环境可切换本地推理服务。

四、引用关系图谱

引用图谱是本项目在「学术算法与图谱挖掘创新」上投入最多的部分。难点不在画图，而在数据：参考文献条目是 PDF 里最难解析的文本之一。

4.1 从 PDF 到图谱的三道关口

4.1.1 第一关：把参考文献条目切分干净

初版实现里约 3300 条参考文献未能抽出标题，人工抽查发现典型形态：标题被截断成作者姓名的碎片、条目跨栏被切断、跨页条目断成两半。处理策略是「多信号切分」——综合标点、年份位置、作者块长度、缩进与续行标记，而不是依赖单一正则。

4.1.2 第二关：把同一篇论文的多个副本合并

库内存在同一篇论文的不同版本与扫描副本。若直接建图，Attention Is All You Need 会被拆成三份，它的被引数和 PageRank 都会失真。系统按「标题 + arXiv 号 + 元数据」交叉核验后合并为单一逻辑节点，实测 1 个节点被合并，其中 arxiv:1706.03762（4 份）。合并不删除任何 PDF 文件。

4.1.3 第三关：三层匹配，宁缺毋滥

层	匹配依据	置信度	出边
1	arXiv 号（归一化前缀、空格、版本号）	最高	是
2	完整规范化标题（大小写/空白/标点/Unicode 全归一）	高	是
3	标题近似（相似度 ≥ 0.92）	中	是
—	相似度落在 0.75~0.92 之间	低	否，记入待人工核验

这条「相似度落在中间区间就不出边」的规则是刻意设计的。标题近似匹配最大的风险是误连不同论文，产生虚假的学术关系。宁可少几条边、把可疑的列出来给人看，也不要让图谱好看但结论错。当前图规模：36 个逻辑节点、189 条引用边。

4.2 图谱挖掘指标

三个指标必须分开统计，否则会得出错误结论，这也是数据里最容易踩的坑：

指标	含义	为什么要区分
引用边数量	入边总数	同一篇论文可能被多篇多次引用
被引论文数量	有多少篇不同的论文引用了它	反映传播广度
不同引用论文数量	去重后的引用方个数	与上项区分，避免重复计数

PageRank 在去重后的逻辑节点上计算，不经过副本重复加权。此外还实现了中介中心性（找跨社区桥接论文）与 Louvain 社区发现。值得注意的是这些算法全部由项目自己用标准库实现，刻意不依赖 networkx / scipy——依赖越少，他人复现时踩坑越少。

4.3 三种图谱挖掘产物

- 基石节点：PageRank 高但入度不高，说明被少量关键工作引用、影响超出直接连接范围。
- 自动综述：基于图结构与元数据生成研究现状概览，不依赖大模型臆测。
- 前沿探索：三路信号——新近且被引、跨社区桥接、未被接入引用网。

五、写作助手

5.1 框架生成

见 3.2.3 的候选集约束生成。补充一个细节：本地论文的作者名单原本读不到，因为论文清单里只有作者数的计数字段，没有名单本身。修复方式是回到细粒度解析结果里按文件名索引作者，使参考文献能列出真实作者。超过 6 位作者按 IEEE 惯例截断为前 6 位加 et al.，并修掉了 et al. 与句号重叠成双句点的问题。

5.2 架构图渲染

提供三种渲染器，覆盖不同投稿场景：

渲染器	产物	适用场景	实现要点
Mermaid	PNG	快速预览、文档内嵌	语法接近自然语言，模型不易写错
Graphviz	PNG	黑白方框，最像论文图	注入 CJK 字体，否则中文全是方框
TikZ	PNG	LaTeX 投稿，原生矢量	编译失败只回喂错误重试一次

工程里有三个真实的坑值得记录：Windows 上 shutil.which 会优先命中 npm 的无扩展名脚本导致「不是有效的 Win32 应用程序」，改为按 .cmd/.exe 顺序手动查找；mermaid-cli 12 移除了 -w 参数改为 -s；Graphviz 默认字体不含中文字形，需在生成的代码里注入 fontname。

5.3 数据可视化

CSV 上传后自动做统计画像、选图型、出交互预览与定稿 PNG。三个决策值得说明：

- 双路输出：ECharts 供浏览器交互预览，matplotlib 供投稿导出。服务的是论文流程的不同环节。
- 对数轴阈值取 100 倍：参数量从 0.34B 到 540B 跨度 1600 倍，线性轴上小模型全挤在原点；而准确率 86~93 不跨数量级，用对数轴反而误导。
- 雷达图逐列归一化：各维量纲不同，不归一化会让大数值维度吃掉整张图。

Caption 生成有一条纪律：只给模型统计摘要，不给图。否则模型会描述出图上并不存在的趋势。实测生成的 caption 里所有数字都来自统计摘要。

5.4 中译英

与阅读页的英译中方向相反，提示词完全另写。目标是译文能直接塞进 IEEE/ACM 模板，因此硬性要求：只输出译文、正式学术语域、术语一致、长句拆短、数字单位与引用标记原样保留、时态遵循学术惯例、不增不减。支持术语表（每行「中文=English」），分块翻译时把上一块末尾作为上下文，保持术语连贯。

六、工程质量

6.1 测试

340 条 pytest 用例覆盖解析、检索、图谱、写作、接口与边界条件。一条重要的测试哲学：只测确定性逻辑，不测大模型输出质量。

例如图谱测试里，若断言「某条文献应被匹配到某篇论文」，正则或阈值任何一侧微调都会让测试红掉，但功能可能更好。因此测试聚焦在可判定的部分：引用校验、去重逻辑、分块边界、选图规则、字体注入是否发生。而「两篇标题很像时该不该连边」这种规则断言，则必须测——它是正确性问题。

写测试时还踩到一个真实教训：构造「相似但不同」的用例必须确认相似度真的低于阈值。最初用「A Survey of Large Language Models」对「Survey of Large Language Models」做拒连用例，实测相似度 0.9688，高于默认阈值 0.92，系统本来就会连边——用例本身就是错的。

6.2 依赖治理

本项目遇到的最大的一个工程问题：依赖清单与代码脱节。实际 import 的 8 个包（pymupdf、pdfplumber、networkx、matplotlib、pandas、numpy、tokenizers、rapidocr）一个都不在清单里，意味着任何人克隆后执行 pip install -r requirements.txt 都起不来服务。

修复的同时加了自检脚本 scripts/check_deps.py：用 AST 扫描 app/ 的真实 import，与清单比对，缺失即退出码 1，已挂进 CI。这样同类问题不会再悄悄发生。同时删掉了 6 个从未被 import 的包（redis、sqlalchemy、pymysql、aiohttp、sse-starlette、tiktoken），减少安装体积与攻击面。

6.3 容器化

Dockerfile 基于 python:3.11-slim（选 Debian 而非 alpine：chromadb、onnxruntime、matplotlib 依赖 glibc 与预编译 wheel，alpine 的 musl 会让 pip 现场编译）。额外装了 graphviz、libgomp1 与 fonts-noto-cjk——最后一个是为了让容器内导出的图表中文不变方框。.dockerignore 把构建上下文从 2.1 GB 压到 27 MB。

6.4 安全

- API 密钥只通过环境变量读取，.env 已加入 .gitignore 并在提交前逐项人工核查。

- 原始 PDF 只读，任何功能都不修改用户文件。
- 索引与图谱均可从原始 PDF 重建，损坏不影响数据。

七、创新点

7.1 候选集约束生成：让幻觉在结构上不可能

常规抗幻觉是「生成后校验、不合格重试」。本项目把约束前移到生成结构本身：文献池由代码提供、引用只能写 key、参考文献由代码生成。模型即使想编，也无处下手。这一思路同时用于问答（证据不足则拒答）与写作（池空则不生成）。

7.2 引用图谱的三层匹配与「宁缺毋滥」

多数引用图谱工具追求边的数量，本项目主动放弃一部分边：相似度落在 0.75~0.92 之间的一律不出边，转人工核验。理由是这个图谱要用于学术判断，一条假边的破坏力远大于十条真边的价值。

7.3 多智能体：显式命名而非堆叠调用

在「像不像多智能体」与「演示能不能跑完」之间，本项目选择后者：把处理阶段显式建模成四角色并回显可核验产物，既让用户与评委看清系统做了什么，又不为形式牺牲响应速度。

7.4 工程细节上的多个「真实坑」

项目的不少细节来自真实调试而非设计推演：Graphviz 中文豆腐块、mermaid-cli 在 Windows 上的 shutil.which 陷阱、小数轴阈值、雷达图量纲归一、pdf.js 文本项之间无分隔符导致句边界定位失败、以及「融合分」与「重排分」量纲混淆。这些已写入 README 的技术决策记录。

八、验收与复现

8.1 快速启动

```
# 1. 安装依赖
pip install -r requirements.txt

# 2. 配置密钥（复制模板后填入自己的 Key）
cp .env.example .env

# 3. 启动服务
uvicorn app.main:app --host 127.0.0.1 --port 8000

# 4. 浏览器打开
http://127.0.0.1:8000/ui/index.html
```

8.2 自检脚本

脚本	作用
scripts/check_all.py	环境、索引、模型、Git、依赖一键体检
scripts/check_deps.py	依赖清单与代码 import 一致性检查
scripts/build_structure.py	重建细粒度解析结果
scripts/build_graph.py	重建引用图谱
scripts/build_index.py	重建向量索引
scripts/verify_official5.py	官方 1.1~1.4 条目验收脚本

8.3 质量门禁

```
ruff check .                  # All checks passed!
pytest tests -q               # 369 passed
python scripts/check_deps.py # ✓ 依赖清单覆盖全部 import
```

以上三项已配置在 .github/workflows/ci.yml 中，每次推送自动执行。

九、已知局限与后续工作

局限	现状	后续方向
低置信度匹配需人工核验	48 条落在阈值之间，未入图	引入引用上下文语义判断，降低误判
多智能体未做模型级协作	四阶段显式命名，非独立模型对话	在延迟允许时引入 Planner 独立推理
外部库依赖网络	arXiv / Crossref 查询需联网	增加本地元数据缓存与降级路径
容器内渲染器不全	只装了 Graphviz	按需提供 Mermaid/TikZ 的镜像变体
重排模型需手工放置	data/models/ 不入库，缺失时降级	提供下载脚本与校验和

上述局限均已记录在仓库 README 与 docs/ 下的交接文档中，没有以「已实现」的方式掩盖。其中第一条与第五条是后续最优先处理的两项。